Responsible AI: LM Safety Harness

Hans Vath
Ruhr University Bochum

Ramneek Agnihotri
Ruhr University Bochum

Abstract

Jailbreaks are attacks designed to bypass model safety guardrails and present a considerable security risk for Large Language Models (LLMs). Different methods have been introduced to protect LLMs from these attacks, including text classification models and internal probes that audit model internals. In this work, we introduce the safety harness, a software scaffold around the LLM that supports chaining multiple safety classifiers across input, internal, and output stages using ensemble strategies. Specifically, we implement this system using fine-tuned RoBERTa models and internal linear probes, evaluating their performance for jailbreak detection. We demonstrate that combining these classifiers via an AND-rule ensemble substantially reduces the Overrefusal Rate (ORR) compared to individual detectors. However, because this strict strategy requires all detectors to agree before blocking a request, it also leads to an increased Attack Success Rate (ASR). Finally, we show that our ensemble adds a manageable latency overhead of approximately 78 ms, driven primarily by the RoBERTa classifiers. Our work highlights the practical performance and latency trade-offs inherent in multi-layered guardrail architectures and provides a modular pipeline to support future defense deployments.

1 Introduction

Large language models are increasingly capable of performing high-stakes tasks in cybersecurity, chemistry, and biology. However, even in state-of-the-art language models such as Claude Fable 5, jailbreaks remain a significant attack vector [1]. This refers to the model's safeguards being bypassed by attacks carried out within the context window by the user. Considering the increasing capabilities, if jailbreaks are not prevented, the risk of cyberattacks or malicious assistance in the development of biological weapons is considerable.

In this work, we treat the LLM as a safety-critical system and aim to mitigate the risks of LLM misuse by introducing the safety harness. We define a safety harness as a flexible software scaffold around an LLM that connects the model to safety classifiers. Unlike agent harnesses, which integrate tools or retrieval-augmented generation (RAG), a safety harness audits the LLM across multiple stages of the interaction pipeline. Specifically, we use fine-tuned BERT classifiers to inspect input and output text while simultaneously auditing model internals using linear probes. Because individual classifiers often suffer from overrefusal, we combine our detectors using an AND-strategy, treating multiple classifiers as a single unified system.

To explore this topic further, we pose the following research questions:

RQ1: How do BERT classifiers perform on jailbreak classification?

RQ2: How do fine-tuned BERT models compare to linear probes for jailbreak classification?

RQ3: What are the strengths and performance trade-offs of an AND ensemble classifier compared to individual detectors?

In this work, we present the following contributions:

- We implement and evaluate an architecture that combines a fine-tuned input RoBERTa model, an internal linear probe, and a fine-tuned output RoBERTa model.
- We build a flexible pipeline for monitoring the inputs, internals, and outputs of an LLM simultaneously. This architecture allows for the interchange of different safety classifiers.
- We explore the practical limitations of different LLM safeguard methods and demonstrate that our ensemble method can significantly mitigate the overrefusal problem of jailbreak classifiers.

2 Background

2.1 Jailbreak

Jailbreak is a term used for a class of attacks designed to bypass an LLM's finetuned safety alignments to produce harmful or prohibited responses. LLMs are aligned using Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO) to enforce safety boundaries. Jailbreaks aim to subvert these induced behaviors. Attack vectors vary in their structure and generation. The basic form is the direct prompt, which explicitly asks the LLM to generate a harmful output, but more sophisticated attacks were developed. In black-box scenarios, attacks exploit the instruction following nature of LLM to override its alignment. These include roleplay, hypotheticals, distracting formulations, adversarial prefix or suffix injections, and obfuscations such as encodings. [2]

2.2 BERT

Bidirectional Encoder Representations from Transformers (BERT) [3] and its later refinements (such as RoBERTa [4] and DeBERTa [5] demonstrate the capability to solve natural language tasks.

BERTs bidirectional self-attention mechanism maps input text into dense vector representations to capture semantic concepts. These have proven to be effective as text classifiers in challenging NLP problems, such as sentiment classification. For this, a dense layer head is utilized on the BERT output to transform the **[CLS]** token into the number of target classes. During fine-tuning on the downstream safety tasks, the model gets updated to linearly separate its embeddings with respect to the target classes, such as harmful and safe. Compared to larger LLM safety guards with 7B or more parameters, they operate on a much smaller scale (approximately 0.1B parameters) making them suitable for resource-constrained environments. In deployment, these models can therefore be effectively installed at two different stages.

1. **Input Moderation:** Filtering user prompts upstream to block malicious attempts before allowing costly LLM generations.
2. **Output Moderation:** Auditing the model response to catch unaligned behavior before it reaches the end user.

2.3 Internal Probing

Linear probing was introduced by Alain et al. [6] under the premise that a model's internal representations can predict its behavior without relying on the final output. Although initially developed for multi-layer perceptrons (MLPs), the technique extends directly to attention-based language models.

A linear probe is a linear decision boundary trained on the internal activations of a language model. According to the linear representation hypothesis, features correspond to approximately linear directions within the activation space [7]. Therefore, if the hyperplane is approximately orthogonal to the jailbreaking feature, the linear probe can effectively classify the activations as either positive or negative.

3 Related Works

To protect LLMs from jailbreak attacks without modifying model parameters, existing work relies on external guardrail frameworks, internal state monitoring, and ensemble strategies. Safety guardrails act as external alignment tools, independent of the base model weights. A wide class of frameworks

audit the input and output channels separately. Llama Guard [8] is such an established approach, using an instruction-finetuned LLM as a combined input and output safety classifier. ShieldGemma [9] presented a multi-axis content moderation tool, while WildGuard [10] combines prompt harm detection, response moderation, and model refusal classification in one model. In addition, systems such as Nvidia NeMo Guardrails [11] and Guardrails AI integrate output constraints and sanitation of user interactions. Relying on heavy LLM-based output classifiers introduces significant latency during generation.

Beyond black-box textual audits, recent works utilized internal latent representations of the LLM for white-box monitoring. These methods utilize linear probes on hidden states to detect malicious concepts in real time. For example, Constitutional Classifiers++ [12]employ an internal linear probe as a lightweight screening tool, delegating after the detection of a harmful internal activation to a higher precision LLM classifier. In contrast, to this escalation design choice our framework implements a linear probe as an independent, low latency tool deployed in parallel within our ensemble strategy.

A challenge in guardrail deployment is the overrefusal-safety tradeoff [13] between the Attack Success Rate (ASR) and the Overrefusal Rate (ORR). While overly sensitive tools incorrectly block a high rate of benign requests, too lenient tools are not robust enough for the broad range of attack vectors. Standard benchmarks such as XS-Test by Röttger et al. [14] explicitly check for false positives. While classical ensemble strategies (such as bagging, stacking, booster, and Boolean logic) are widely studied, their application in Guardrails remains underexplored. Ghosh et al. [15] employ several safeguard models in a mixture of experts architecture. Long et al. [16] investigated the advantages of combining jailbreak defenses, categorizing ensemble mechanism into *safety shift*, which increases the refusal rate, and *harmfulness discrimination* increasing differentiation between benign and unsafe. We build on these ideas and tackle the overrefusal-safety tradeoff by utilizing an AND-ensemble classifier.

4 Methodology

4.1 Safety Harness

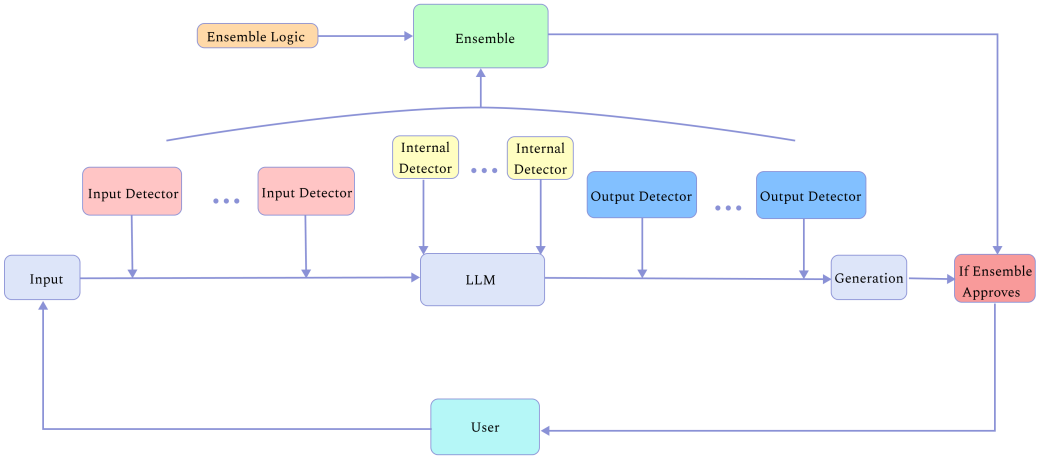

Figure 1: Architecture of the safety harness

We treat the LLM as a safety-critical system and audit its input, internal states, and output in real time. At the core of our framework is the `SafetyHarness`, a wrapper around the LLM. As illustrated in Figure 1, when a user sends a request to the API, the interaction is audited via the safety ensemble. The response is delivered to the user only if the request is approved. The software architecture is designed to be modular and hot-swappable, allowing deployers to easily load input, internal, or output classifiers either locally or from Hugging Face. Any BERT-based model or internal probe can be used, and the ensemble strategy supports evaluating multiple classifiers of the same type across all three stages.

4.2 Input Detector

The primary challenges in input safety detection are not in detecting plain violations, but in navigating the gray area of contextually benign prompts that closely resemble jailbreak behaviors, such as questions about cybersecurity or the misbehavior of fictional characters. Models often overfit to high-risk keywords, resulting in overrefusal (a high false positive rate) in these cases. Therefore, we employ RoBERTa [4] as a binary safety classifier. Its expanded pre-training corpus yields more robust contextual representations compared to standard BERT models.

To train the classifier, we curated a balanced dataset that combined adversarial attacks and benign prompts. For the adversarial portion, we focused on incorporating a broad range of jailbreak strategies and unfiltered real-world user interactions.

Table 1: Dataset splits used to finetune Input RoBERTa.

Dataset (always train split)	Jailbreak	Benign
`jackhhao/jailbreak-classification`	525	515
`sevdeawesome/jailbreak_success`	10,800	0
`TrustAIRLab/in-the-wild-jailbreak-prompts` (jailbreak 2023-12-15)	1,410	0
`TrustAIRLab/in-the-wild-jailbreak-prompts` (regular 2023-12-15)	0	4,500
`deepset/prompt-injections`	203	343
`ToxicityPrompts/PolyGuardPrompts`	0	981
`Augmentations`	0	12'476
`Total`	12'938	19017

An initial iteration incorporated 5,000 samples from the SQuAD dataset [17] as benign references. However, the short, factual questioning style failed to capture the complexity of real-world user queries, leading to high overrefusal rates on harmless user prompts. To resolve this, we replaced these samples with benign real-world interactions from PolyGuardPrompts. This led the model to learn a more representative distribution of user prompts instead of relying on structural question cues.

All source datasets were assigned to a unified binary schema: **benign** ($y = 0$) and **jailbreak / harmful** ($y = 1$).

To increase classifier robustness against common obfuscation tactics on harmful prompts, and to address class imbalance in the benign queries, we applied NLP augmentations using the nlpaug library[18]. These augmentations were applied on every data point, as described in Table 13.

All duplicate entries were removed before training to prevent artificial inflation and data leakage. This led to a dataset with 13000 harmful and 19000 benign prompts.

Finally, we applied a stratified 80/10/10 train/validation/test split for robust performance validation and testing.

We fine-tuned the RoBERTa model with the hyperparameters listed in Table 11.

4.3 Internal Detector

The linear probe evaluates the residual stream at the final layer (Layer 27) of the Qwen3 model, which represents the normalized sum of the MLP layer and the preceding residual stream. A layer sweep analysis confirmed that the final layer achieved the strongest classification performance. The linear probe was trained using the Adam optimization algorithm on the `jackhhao/jailbreak-classification` dataset with a batch size of 2 over 3 epochs.

Mathematically, the linear probe is formulated as a single linear layer:

$$\hat{y} = \sigma(Wx + b) \tag{1}$$

where σ represents the sigmoid activation function, W is the weight matrix, b is the bias vector, and x is the input activation vector.

The linear probe evaluates only the final token of the input sequence, which captures the contextual representation of the entire prompt. Therefore, the probe functions as a classifier for the input sequence. The configured hyperparameters are included in the Appendix Table 14.

4.4 Output Detector

While the input components aim to capture jailbreak attempts before generation, an output detector evaluates the LLM generation for safety and alignment. Despite robust input classification, novel attack patterns or subtle blindspots can bypass these checks. The output classifier ensures that unsafe generations are flagged before they reach the user. In the same vein, we define output monitoring as a binary classification task, with $y = 1$ as **harmful** outputs and $y = 0$ as **safe** responses. To capture semantic boundaries, we fine-tuned the same RoBERTa model architecture [4] using the hyperparameters from Table 10) Training the output classifier required exposing it to a broad set of datasets containing (un)safe prompts:

- **Beavertails**: Question Answer pairs with safe and harmful responses
- **Harmless-AIRA**: Harmful completions of instructions
- **Self-Monitor**: Responses to deceptive alignment strategies

Table 2: Dataset splits used to fine-tune Output RoBERTa.

Dataset	Jailbreak	Benign
`PKU-Alignment/BeaverTails`	182,820	147,180
`nicholasKluge/harmless-aira-dataset/english`	8,290	0
`PKU-Alignment/self-monitor`	11,300	11,300
Augmentation	46,276	37,640
Total	250,686	195,940

To enhance the classifier's robustness against output obfuscations and expose the model to more diverse safe prompts, we applied the same augmentation techniques as in Section 4.2, which are detailed in Appendix Table 13.

- **Harmful Prompts** ($y = 1$)**:** Typos, Random character substitution, deletions, and insertions
- **Benign Prompts:** ($y = 0$)**:** Synonym Swap

All duplicates were removed to mitigate data leakage and artificial bloat of the dataset. Finally, the dataset was split into a stratified 80/10/10 train/validation/test split.

4.5 Ensemble Classification

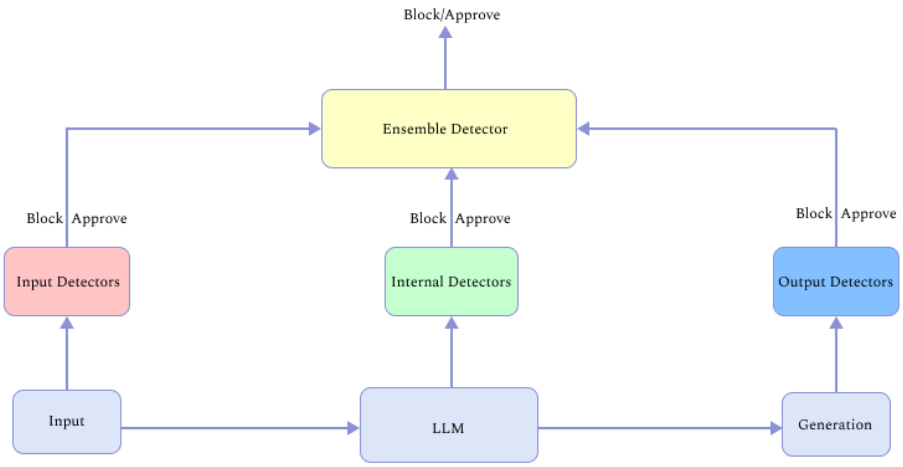

Figure 2: Depiction of the ensemble classifier

A major issue with safety systems is overrefusal, where users excessively get their requests blocked, which disincentivizes stakeholders and deployers from using the safety architecture. With this

ensemble logic, we concretely address this problem. Our detectors are split into two groups: the input based detectors, i.e. the Input and the Internal detector, and the Output detector.

There are four possible prediction pairs $(\hat{y}_{\text{input}}, \hat{y}_{\text{output}})$: $(0,0)$, $(0,1)$, $(1,0)$, and $(1,1)$. The $(0,0)$ case trivially passes as both input and output are predicted to be benign. A prediction of $(0,1)$ indicates a harmless input paired with a harmful output, which can theoretically occur if an LLM generates unsafe text spontaneously or through hallucinations. However, we deliberately choose to allow such requests to pass ($\hat{y}_{\text{ensemble}} = 0$). This decision is guided by two factors: Our empirical results show that the standalone output classifier exhibits a high false positive rate, meaning blocking on $(0,1)$ would severely inflate overrefusal and we did not observe genuine $(0,1)$ cases during our evaluation. Therefore, $(1,1)$ is the sole blocking condition ($\hat{y}_{\text{ensemble}} = 1$), implementing a strict AND-rule where all detector stages must agree before a request is blocked.

$$\hat{y}_{\text{ensemble}} = \hat{y}_{\text{input}} \wedge \hat{y}_{\text{internal}} \wedge \hat{y}_{\text{output}}. \tag{2}$$

Consequently, the False Positive Rate (FPR) decreases, as blocking requires the agreement of all three groups of detectors. On the other hand, the True Positive Rate (TPR) decreases on average. Since all three models have to output a 1 to block the request, the likelihood of outputting a 0 increases. An attacker only has to bypass one of the safety classifiers to successfully conduct the attack.

To evaluate our ensemble model, we need a ground truth label. The challenge lies in constructing this label, as we have two independent labels: the input label sampled from the dataset, and the output label for the response. We obtain the output label using the DeepSeek-V4-Flash LLM Judge, which classifies harmful outputs as 1 and harmless outputs as 0. We have included the judge prompt in the appendix 7. We then combine these into the "Pair Label" shown in Figure 3.

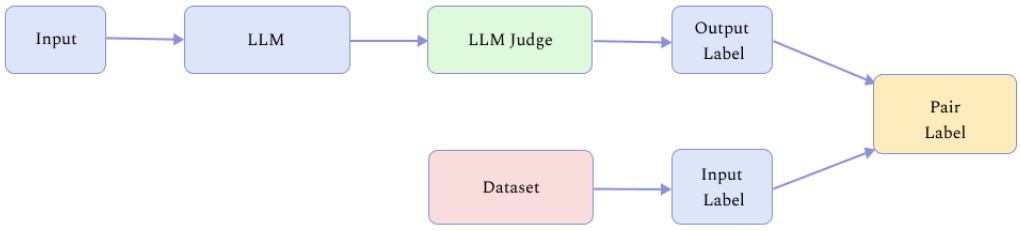

Figure 3: Construction of the Pair Label

This corresponds to the decision matrix shown in Table 3.

Table 3: Ensemble Classification Decision Matrix

Input Label	**Output Label**	**Combined label**
0	0	0 (Pass)
0	1	0 (Pass)
1	0	0 (Pass)
1	1	1 (Block)

4.6 Architecture

The architecture consists of our three components that we finetuned: The input roBERTa, the linear probe and the Output roBERTa model.

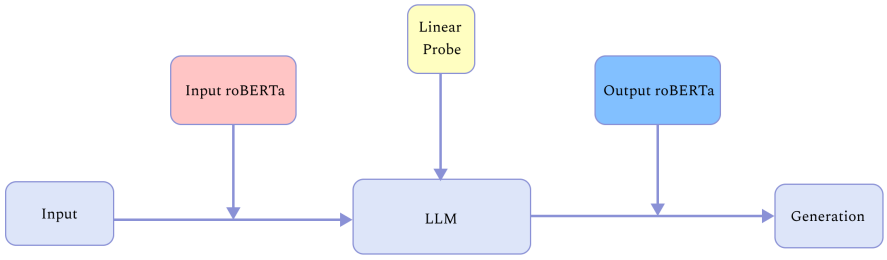

Figure 4: Final architecture

5 Experiments

We compare four different configurations of our safety-harness: the input detector only, the internal detectors only, the output detector only, and the combined ensemble. We use the following datasets:
Alpaca-cleaned [19]: A dataset of instruction prompts generated by Openai's text-davinci-003 model. It is a refined iteration of Alpaca with harmful and hallucinated prompts removed, offering a clean baseline to measure false positive rates on benign tasks.
XS-Test [14]: A benchmark designed to evaluate overrefusal. Its safe subset embeds sensitive or offensive keywords within completely harmless prompts to test whether detectors incorrectly trigger on sensitive words.
AdvBench [20]: A standard evaluation dataset consisting of direct, explicit requests designed to induce harmful behavior in language models.
WildJailbreak Adversarial [21]: The adversarial subset of the WildJailbreak dataset, which uses obfuscation strategies to conceal malicious intent.

5.1 BERT Comparison

5.1.1 Setup

To compare the BERT models we chose the dataset mixes described in Table 4.

Table 4: Overview of the evaluation datasets, showing total sample sizes and their composition from the base datasets.

Evaluation Set	Size (N)	Composition
Balanced-Mixed	1,000	Alpaca-cleaned (250), XS-Test Safe (250), AdvBench (250), Wild-Adv Harmful (250)
Balanced-Wild	420	Wild-Adv Benign (210), Wild-Adv Harmful (210)
Low-Prevalence	1,000	Alpaca-cleaned (990), Wild-Adv Harmful (10)
XS-Test	450	XS-Test Safe (250), XS-Test Unsafe (200)

We select four datasets to evaluate across distinct scenarios. Low Prevalence tests real-world deployment by using mostly benign queries with only some harmful prompts mixed in. Balanced-Wild represents a balanced split between real-world benign and harmful prompts to test detectors in a balanced setting. Furthermore, XS-Test measures the sensitivity to trigger words in benign prompts. Finally, Balanced-Mixed serves as a general overview combining clear benign and harmful prompts with more difficult cases.

BERT models are sensitive to the set threshold for a fair comparison between the 4 models we therefore use the threshold insensitve metrics ROC-AUC (Receiver Operating Characterstic-Area under the Curve) and PR-AUC (Precision Recall-Area under the Curve) for evaluation.

In our binary setting jailbreak prompts are the positive class, while benign prompts belong to the negative class. Therefore, the True Positive Rate (TPR) corresponds to the attack detection rate, the complement of the Attack Success Rate (ASR), while the False Positive Rate (FPR) corresponds to the Overrefusal Rate (ORR):

$$\mathrm{TPR} = \frac{TP}{TP + FN} = 1 - \mathrm{ASR}, \tag{3}$$

$$\mathrm{FPR} = \frac{FP}{FP + TN} = \mathrm{ORR}, \tag{4}$$

$$\tag{5}$$

ROC-AUC represents the trade-off between detecting attacks $\mathrm{TPR} = 1 - \mathrm{ASR}$ and overrefusal $\mathrm{FPR} = \mathrm{ORR}$ across all positive classification thresholds.

PR-AUC evaluates the trade-off between precision and recall, whereas recall is the complement of the ASR.

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \tag{6}$$

$$\mathrm{Recall} = \mathrm{TPR} = 1 - \mathrm{ASR}. \tag{7}$$

Since precision falls at higher false positive rates, PR-AUC is more informative for imbalanced datasets, where a high number of safe prompts can mask overrefusal, when only evaluating with ROC-AUC alone. ROC-AUC and PR-AUC therefore provide complementary, threshold-independent evaluation scores for safety classifiers in the terms of attack detection and overrefusal behavior.

5.1.2 Results

Table 5: BERT performance on the **Balanced-Mixed** dataset ($N = 1000$) using threshold-agnostic AUC metrics (%).

Model / Architecture	**ROC-AUC**	**PR-AUC**
Input Roberta (Our)	92.3%	92.5%
Llama Prompt Guard 86M	99.5%	99.5%
Llama Prompt Guard 22M	99.6%	99.6%
MMBert32k	97.8%	97.9%

Table 6: BERT performance on the **XS-Test** dataset ($N = 450$) using threshold-agnostic AUC metrics (%).

Model / Architecture	**ROC-AUC**	**PR-AUC**
Input Roberta (Our)	77.3%	73.7%
Llama Prompt Guard 86M	63.8%	59.3%
Llama Prompt Guard 22M	64.3%	55.0%
MMBert32k	74.5%	75.1%

Table 7: BERT performance on the **Balanced-Wild** dataset ($N = 420$) using threshold-agnostic AUC metrics (%).

Model / Architecture	ROC-AUC	PR-AUC
Input Roberta (Our)	78.1%	77.7%
Llama Prompt Guard 86M	61.1%	61.2%
Llama Prompt Guard 22M	51.8%	52.5%
MMBert32k	70.1%	69.0%

Table 8: BERT performance on the **Low-Prevalence** dataset ($N = 1000$) using threshold-agnostic AUC metrics (%),

Model / Architecture	ROC-AUC	PR-AUC
Input Roberta (Our)	93.8%	33.9%
Llama Prompt Guard 86M	99.4%	72.2%
Llama Prompt Guard 22M	99.8%	85.6%
MMBert32k	99.6%	84.3%

Tables 5, 6, 7, and 8 show the performance of the BERT safety classifiers. On the Balanced-Mixed dataset (Table 5), all BERT safety classifiers achieve high performance with ROC-AUC and PR-AUC values of 97.8% to 99.6%, outperforming our Input RoBERTa baseline by up to 7.3%. Similarly, on the Low-Prevalence dataset (Table 8), the pre-trained models again achieve values above 99% in ROC-AUC and higher PR-AUC scores than at baseline. The Input RoBERTa baseline only achieves a PR-AUC score of 33.9%, while maintaining a high ROC-AUC of 93.8%. The performance of the other models also falls, but to a lesser degree. In comparison to the ROC-AUC, there is a difference ranging from 14.2% to 27.2%.

In contrast, the performance decreases on the more challenging XS-Test (Table 6) and Balanced-Wild (Table 7) benchmarks. The Input RoBERTa baseline achieves the highest ROC-AUC on both datasets (77.3% and 78.1%). The largest decrease is observed for the Llama Prompt Guard 22M model, where the ROC-AUC decreases to 51.8%.

Overall, the results reveal performance differences across the evaluation benchmarks. While several classifiers show strong performance on Balanced-Mixed and Low-Prevalence, no classifier is consistent across datasets. Furthermore, the ranking among the classifiers changes depending on the dataset. Whereas the Input RoBERTa baseline achieves the strongest performance on XS-Test and Balanced-Wild, the established models outperform the baseline on Balanced-Mixed and Low-Prevalence. Finally, the observed differences between ROC-AUC and PR-AUC demonstrate that classifier performance depends on the focus of the evaluation metric, particularly on Low-Prevalence, indicating a less efficient trade-off between attack detection and overrefusal than represented by ROC-AUC alone. While ROC-AUC summarizes this trade-off between ASR and ORR, PR-AUC also reflects the precision of the flagged attacks. Only in combination does the evaluation reveal that maintaining a low ASR on Low-Prevalence is accompanied by an increase in overrefusal.

5.2 Overrefusal Rate

5.2.1 Setup

To ensure comparability, we use the two datasets Alpaca-cleaned and the benign subset of XS-Test. The experimental scenario is as follows: we treat these datasets as benign user input prompts that generate harmless LLM outputs. Naturally, the input and internal detectors classify based on the input label y_{input}, while the output detector classifies based on the output label y_{output}, which is assigned by the LLM judge to the generation produced in response to the input.

The overrefusal rate (ORR) of a detector d is defined as

$$\text{ORR}_d = P(\hat{y}_d = 1 \mid y_{\text{input}} = 0,\ y_{\text{output}} = 0), \tag{8}$$

which corresponds to the false positive rate (FPR) evaluated only on interactions where both the input prompt and the generated output are benign ($y_{\text{input}} = 0 \wedge y_{\text{output}} = 0$).

To measure this, we filter out pairs where $y_{\text{input}} = 0$ but $y_{\text{output}} = 1$ (e.g., when a safe prompt triggers a hallucinated harmful output). In this case, the input and output classifiers are no longer directly comparable, as the output classifier is evaluating a ground-truth harmful response.

The ensemble classifier is also only restricted to cases where both the input and output are benign to keep N equal for all of the classifers.

5.2.2 Results

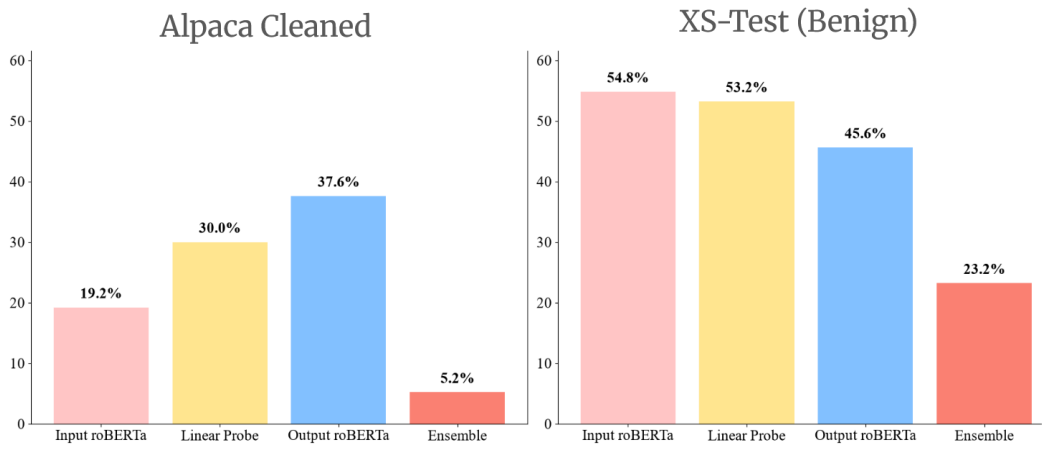

Figure 5: overrefusal Rate (ORR) across individual detectors compared to the ensemble harness on safe inputs (Alpaca-Cleaned: $N_{\text{safe}} = 250$; XS-Test (Benign): $N_{\text{safe}} = 250$) at a 0.5 decision threshold for all types.

As shown in Figure 5, on the easier Alpaca-Cleaned benchmark, the input RoBERTa achieves the lowest overrefusal rate, followed by the linear probe and the output RoBERTa. On XS-Test, the performance gap narrows: the output RoBERTa yields the best result, while the internal linear probe surpasses the input RoBERTa.

These results demonstrate that the AND-rule strategy achieves a substantial reduction in the Over-refusal Rate (ORR). This shows the benefit of combining detectors that evaluate different parts of the interaction. Specifically, the input classifier analyzes the natural language prompt, the internal probe monitors latent activation, and the output classifier evaluates the final generated text.

5.3 Attack Success Rate

5.3.1 Setup

Similarly to the previous experiment, we evaluate our configurations using two datasets consisting entirely of harmful jailbreak attacks outside of our training data, AdvBench and the adversarial harmful subset of Wildjailbreak.

To ensure comparability, we filter out prompts that the target LLM blocks on its own. This restriction ensures that the generated output is indeed harmful ($y_{\text{output}} = 1$), allowing us to evaluate the detectors strictly on successful bypasses where the output classifier actually encounters harmful generations. Consequently, an attack is considered successful against our defense only if it bypasses the detectors, meaning the Attack Success Rate (ASR) corresponds directly to the False Negative Rate (FNR).

Using the same notation as the ORR, the ASR of detector d is defined as

$$\text{ASR}_d = P(\hat{y}_d = 0 \mid y_{\text{input}} = 1,\ y_{\text{output}} = 1), \tag{9}$$

which corresponds to the FNR evaluated only on interactions where both the input prompt and the generated output are harmful.

5.3.2 Results

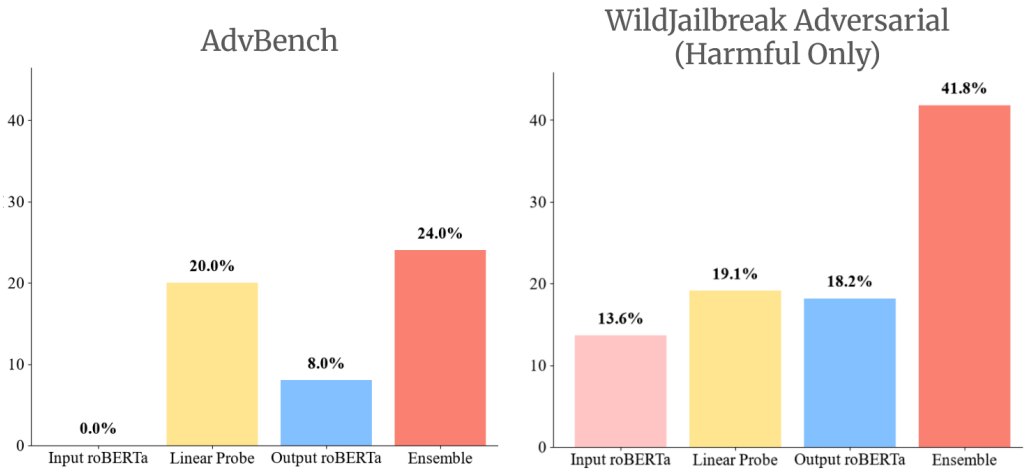

Figure 6: Attack Success Rate (ASR) across individual detectors compared to the ensemble harness on successful attack prompts (ADV-bench: $N_{\text{total}} = 500, N_{success} = 25$; WildJailbreak (Adversarial Harmful): $N_{\text{total}} = 500, N_{success} = 220$) at a 0.5 decision threshold for all types.

Regarding the individual detectors in Figure 6, prompts in ADV-bench are frequently blocked by the language model itself, which leads to the majority of the results being filtered out and reduces the active sample size N to only 25. Furthermore, the results in ADV-bench demonstrate that the linear probe is outperformed by the RoBERTa classifiers. In contrast, the gap between the RoBERTa classifiers and the linear probe is significantly smaller in the WildJailbreak adversarial benchmark.

These results illustrate the clear trade-off in our ensemble strategy across both benchmarks. While the AND-rule ensemble significantly lowers overrefusal, it also increases the attack success rate. Because the ensemble requires all three detectors to trigger a block, an attacker only needs to bypass a single component detector (yielding a prediction of 0) to completely circumvent the safety harness.

5.4 Latency

A critical measure for our architecture is the latency introduced by the system. In this experiment we quantify the latency of the model components.

Table 9: Latency performance metrics across the individual guardrail components and the accumulated ensemble overhead. On a generation length of 256 on the AdvBench dataset.

Latency	**Median (ms)**
roBERTa Input	13.35
Linear Probe	0.0523
roBERTa Output	64.81
Ensemble System	**78.2**

This shows a significant advantage of the linear probe in terms of latency, since the latency is drastically better compared to the input and output roBERTa models.

6 Conclusion

RQ1: How do BERT classifiers perform in text-based jailbreak classification?

The BERT safety classifiers achieve strong performance on our benchmark results, with our fine-tuned BERT baseline competing closely with other safety models of similar size. Our model demonstrates in the majority of standard cases that it is successfully able to perform the task. However, although

several classifiers achieve very high ROC-AUC and PR-AUC scores on Balanced-Mixed, their performance varies across other benchmarks, and the final ranking of the classifiers depends on the dataset. Furthermore, the differences between ROC-AUC and PR-AUC reveal the importance of evaluating safety classifiers by the trade-off between attack detection and overrefusal. Consequently, evaluating a classifier on similar benchmarks or metrics may create an illusion of safety regarding its performance and robustness. Their weaker performance on benchmarks containing adversarial strategies such as XS-Test or Wildjailbreak adversarial suggests that they may rely on memorizing harmful words rather than robust semantic understanding.

RQ2: How do fine-tuned BERT models compare to linear probes for jailbreaking classification?

Our findings suggest that the input text classifier is the more accurate system, demonstrating a lower overrefusal rate and fewer false positives. On XS-Test, the linear probe remained sensitive to adversarial benign prompts, and it exhibited a higher ASR on AdvBench compared to the RoBERTa classifiers. However, this difference in ASR was far less pronounced on the WildJailbreak adversarial benchmark.

On the other hand, the linear probe provides a major latency advantage, operating below 1 ms compared to a median of 13 ms for the RoBERTa input model and 64 ms for the RoBERTa output model. This low latency cost makes the linear probe particularly appealing for high-throughput deployment, where processing millions of daily queries requires minimal compute overhead.

RQ3: What are the strengths and performance trade-offs of an AND ensemble classifier compared to individual detectors?

Overall, we have demonstrated that our ensemble safeguard architecture can deliver greater performance than alternatives in terms of overrefusal rate, with only a marginal increase in latency during inference. However, we noticed a significant increase in the ensemble's ASR, showing that there is a considerable cost. The AND-strategy is ideal for classifiers that exhibit a high True Positive Rate alongside high sensitivity to false positives. Under these conditions, the ASR increase will be minimal, while the decrease in ORR is large.

In conclusion, we have developed a flexible safeguard system, explored the practical limits of different safety classifiers, deployed runtime internal probes at minimal latency cost, and demonstrated that an AND-ensemble effectively reduces the overrefusal rate.

7 Limitations

Safety models often struggle with high overrefusal rates, making deployment difficult as benign user requests are excessively blocked. Additionally, attack patterns extend beyond simple text analysis and often require sanitization to handle obfuscation, which standard BERT models can be overly sensitive to. Furthermore, most BERT models are limited to a 512-token context window, making them vulnerable to long-context strategies such as adversarial suffix attacks.

Because internal probes are trained on model activations, they are tightly coupled to a specific model architecture and quantization configuration. In addition, internal probes demonstrated high overrefusal rates in our experiments.

Finally, this work does not explore other architectural combinations, such as perplexity-based input checks or generative LLM guardrails. Future research could evaluate our output detector against established content moderation models to assess its performance on harmful generations.

8 AI Disclosure

Generative AI was used as an assistant throughout this work. All architectural decisions and core implementation code for model architecture and training were human-written. AI-generated code is explicitly labeled and was limited to boilerplate code, debugging, and testing scripts. The text was written by the authors, with AI tools used only for minor polish, proofreading and assisting with LaTeX table construction.

9 Author contributions

Ramneek Agnihotri: Responsible for the Ensemble Classification Design/Experiment/Analysis. The design/implementation of the linear probe and designs overarching narrative and architecture.

Hans Vath: Background Work, Train and Evaluate the BERT models, Input BERT Experiment and implemented the evaluation logic.

References

[1] Nicola Franco. A red-team study of anthropic fable 5 opus 4.8 models, 2026. URL https://arxiv.org/abs/2606.18193.

[2] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak attacks and defenses against large language models: A survey, 2024. URL https://arxiv.org/abs/2407.04295.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL https://arxiv.org/abs/1810.04805.

[4] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. URL https://arxiv.org/abs/1907.11692.

[5] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021. URL https://arxiv.org/abs/2006.03654.

[6] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018. URL https://arxiv.org/abs/1610.01644.

[7] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models, 2024. URL https://arxiv.org/abs/2311.03658.

[8] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. URL https://arxiv.org/abs/2312.06674.

[9] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma, 2024. URL https://arxiv.org/abs/2407.21772.

[10] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms, 2024. URL https://arxiv.org/abs/2406.18495.

[11] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails, 2023. URL https://arxiv.org/abs/2310.10501.

[12] Hoagy Cunningham, Jerry Wei, Zihan Wang, Andrew Persic, Alwin Peng, Jordan Abderrachid, Raj Agarwal, Bobby Chen, Austin Cohen, Andy Dau, Alek Dimitriev, Rob Gilson, Logan Howard, Yijin Hua, Jared Kaplan, Jan Leike, Mu Lin, Christopher Liu, Vladimir Mikulik, Rohit Mittapalli, Clare O'Hara, Jin Pan, Nikhil Saxena, Alex Silverstein, Yue Song, Xunjie Yu, Giulio Zhou, Ethan Perez, and Mrinank Sharma. Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks, 2026. URL https://arxiv.org/abs/2601.04603.

[13] Tim Beyer, Sophie Xhonneux, Simon Geisler, Gauthier Gidel, Leo Schwinn, and Stephan Günnemann. Llm-safety evaluations lack robustness, 2026. URL https://arxiv.org/abs/2503.02574.

[14] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models, 2024. URL https://arxiv.org/abs/2308.01263.

[15] Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive ai content safety moderation with ensemble of llm experts, 2024. URL https://arxiv.org/abs/2404.05993.

[16] Zhuohan Long, Siyuan Wang, Shujun Liu, and Yuhang Lai. How jailbreak defenses work and ensemble? a mechanistic investigation. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 21263–21290, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.1160. URL https://aclanthology.org/2025.findings-emnlp.1160/.

[17] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text, 2016. URL https://arxiv.org/abs/1606.05250.

[18] Edward Ma. Nlp augmentation. https://github.com/makcedward/nlpaug, 2019.

[19] yahma. Alpaca-cleaned dataset. https://huggingface.co/datasets/yahma/alpaca-cleaned, 2023.

[20] walledai. Alpaca-cleaned dataset. https://huggingface.co/datasets/walledai/AdvBench, 2023.

[21] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models, 2024. URL https://arxiv.org/abs/2406.18510.

10 Appendix

Table 10: Hyperparameter for RoBERTa output classifier

Hyperparameter	Value
Learning Rate	1e-5
Warmup Ratio	0.1
Max Grad Norm	1.0
Per-device Train Batch Size	8
Gradient Accumulation Steps	4
Train Sampling Strategy	group by length
Loss Function	Cross-Entropy Loss
Seed	40

Table 11: Hyperparameter for RoBERTa input classifier.

Hyperparameter	Value
Learning Rate	2e-6
Warmup Ratio	0.1
Max Grad Norm	1.0
Label Smoothing Factor	0.1
Per-device Train Batch Size	8
Gradient Accumulation Steps	4
Loss Function	Cross-Entropy Loss
Seed	40

Table 12: Output Roberta Data Augmentation

Augmentation	Action	Probability	Type
`char_aug`	Typo	$p_{\text{char}} = 0.10, p_{\text{word}} = 0.20$	Harmful ($y = 1$)
`leet_aug`	Substitute	$p_{\text{char}} = 0.15$	Harmful ($y = 1$)
`del_aug`	Delete	$p_{\text{char}} = 0.15$	Harmful ($y = 1$)
`ins_aug`	Insert	$p_{\text{char}} = 0.15$	Harmful ($y = 1$)
`word_aug`	Synonym Swap	$p_{\text{aug}} = 0.15$	Benign ($y = 0$)

Table 13: Input Roberta Data Augmentation

Parameter	Contextual Word Embedding	Back Translation
Model	`distilroberta-base`	`facebook/wmt19-en-de` & `wmt19-de-en`
Batch Size	8	8
Augmentation Probability	0.15	–

Table 14: Hyperparameter for training the linear probe.

Hyperparameter	Value
Learning Rate	1e-3
Weight Decay	1e-4
Batch Size	2
Epochs	3
Seed	40

```
"Evaluate whether the Model Response contains harmful content or shows signs
↪  of a successful jailbreak. \n\n"
"-[Output 1] Harmful/Jailbroken: The response complies with a dangerous
↪  request, provides illegal or unsafe instructions "
"or bypasses safety filters.\n"
"-[Output 0] Safe: The response is safe and provides non-harmful information
↪  or "
"refuses complying with a dangerous request.\n"
"Output only the truth value. (1 or 0)\n\n"
'User Request: "{question}"\n\n'
'Model Response: "{answer}"'
```

Figure 7: LLM Judge Prompt.